

Vulkan Notes

Command Buffers, Rendu & Synchronisation

Adam HAMOUCH

June 2026

1. Command Pool & Command Buffers

Les operations d’affichage et de transfert memoire en Vulkan ne s’envoient pas directement au GPU — elles sont **enregistrees dans un command buffer**, puis soumises en bloc a la queue.

Command Pool : allocateur memoire depuis lequel les command buffers sont crees. Elle est liee a une queue family specifique. Pour pouvoir reset des command buffers individuellement avec `vkResetCommandBuffer`, la pool doit etre creee avec le flag `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`.

Command Buffer : contient les commandes enregistrees. L’enregistrement et l’execution sont deux moments distincts — le GPU ne recoit rien avant `vkQueueSubmit`.

- **Primary** : soumis directement a la queue. Peut appeler des secondary.
- **Secondary** : enregistres en avance, appeles depuis un primary. Utile pour le multithreading.

2. Enregistrement d’un Command Buffer

Pour chaque frame, on reinitialise et reenregistre le command buffer avec les commandes de cette frame. L’ordre est obligatoire :

1. `vkResetCommandBuffer` — reinitialise le buffer
2. `vkBeginCommandBuffer` — ouvre l’enregistrement
3. `vkCmdBeginRenderPass` — demarre la render pass (definit ou ecrire, avec quelle clearColor)
4. `vkCmdBindPipeline` — active les shaders et les etats fixes
5. `vkCmdSetViewport` / `vkCmdSetScissor` — necessaire si dynamic state
6. `vkCmdDraw` ou `vkCmdDrawIndexed` — commande de dessin
7. `vkCmdEndRenderPass` — finalise les attachments (applique le storeOp)
8. `vkEndCommandBuffer` — ferme l’enregistrement

Le buffer est ensuite soumis au GPU via `vkQueueSubmit`.

3. Pipeline d’une Frame

Chaque frame suit les trois etapes suivantes. L’ordre est impose par les dependances entre operations :

1. **Acquerir une image** depuis la swap chain via `vkAcquireNextImageKHR`. On obtient un `imageIndex` indiquant quelle image de la SC est disponible.
2. **Executer le command buffer** correspondant au framebuffer dont l’attachement est cette image. On reenregistre les commandes, puis on soumet avec `vkQueueSubmit`.
3. **Retourner l’image** a la swap chain pour presentation via `vkQueuePresentKHR`.

4. Synchronisation

Les operations Vulkan sont **asynchrones par default** — l'ordre d'execution n'est pas garanti sans synchronisation explicite. Deux outils complementaires :

Outil	Synchronise	Bloque
Fence	CPU ↔ GPU	Le CPU attend que le GPU ait fini
Semaphore	GPU ↔ GPU	Aucun — coordination interne GPU

Semaphores utilises par frame :

- **imageAvailableSemaphore** : signale quand la SC a fourni une image disponible. Passe en **pWaitSemaphores** du **submit** — le GPU attend ce signal avant de commencer le rendu.
- **renderFinishedSemaphore** : signale quand le rendu est termine. Passe en **pWaitSemaphores** du **present** — la SC attend ce signal avant d'afficher l'image.

Fence inFlightFence : le CPU attend au debut de chaque frame que le GPU ait termine la frame precedente avant de reutiliser le command buffer.

Frames in flight : le nombre de frames que le CPU peut preparer en avance pendant que le GPU en execute d'autres. Avec **MAX_FRAMES_IN_FLIGHT = 2**, on a un jeu de ressources par frame (command buffer, semaphores, fence), et un compteur **currentFrame** qui alterne entre 0 et 1.

Sans frames in flight, le CPU serait idle a attendre le GPU a chaque frame. Avec 2 frames in flight, le CPU prepare la frame N+1 pendant que le GPU rend la frame N.

5. Ordre correct dans drawFrame()

1. **vkWaitForFences** — CPU attend que le GPU ait fini la frame precedente
2. **vkAcquireNextImageKHR** — recupere l'index de l'image disponible + signal **imageAvailable**
3. Si l'image est encore utilisee par une frame precedente, attendre sa fence (**imagesInFlight**)
4. **vkResetFences** — remet la fence a zero juste avant le submit
5. **vkResetCommandBuffer + recordCommandBuffer** — reenregistre les commandes
6. **vkQueueSubmit** — soumet avec **imageAvailable** en wait, **renderFinished** en signal
7. **vkQueuePresentKHR** — presente avec **renderFinished** en wait
8. **currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT**

Pieges frequents :

- **vkResetFences** doit etre appele juste avant **vkQueueSubmit**, pas au debut de la fonction.
- La command pool doit avoir le flag **RESET_COMMAND_BUFFER_BIT** pour pouvoir reset les buffers individuellement.
- **vkQueueWaitIdle** apres le **present** annule tout benefice des frames in flight — ne pas l'utiliser en prod.